

Bài 1:

1. Mô tả lỗi (bug): Trong cài đặt hàm `union(int p, int q)`, vòng lặp `for` duyệt từ $i = 0$ đến `leader.length - 1`.
 - Điều kiện kiểm tra trong vòng lặp là `if (leader[i] == leader[p])`. Khi vòng lặp chạy đến vị trí $i = p$, điều kiện này thỏa mãn nên `leader[p]` bị cập nhật thành giá trị mới là `leader[q]`.
 - Do `leader[p]` đã bị thay đổi trực tiếp ngay giữa quá trình duyệt mảng, các phần tử nằm sau p thuộc cùng tập hợp với p ban đầu sẽ bị so sánh với giá trị `leader[p]` mới thay vì giá trị `leader[p]` cũ dẫn đến các phần tử này không được cập nhật sang đại diện mới `leader[q]`.
2. Testcase chứng minh:
 - $n = 4$ (mảng `leader = [ 0 , 1 , 2 , 3 ]`)
 - Thực hiện lần lượt các phép toán `union(1,2)` và `union(1,3)`
 - Sau khi thực hiện lệnh `union(1,2)`: mảng trở thành `leader = [ 0, 2 , 2 , 3 ]`. Lúc này phần tử 1 và phần tử 2 đã thuộc cùng một tập hợp với đại diện là 2
 - Sau lệnh `union(1,3)` (với $p = 1$, $q = 3$):
 - + Khi $i = 0$: `leader[0] != leader[1]` -> Không đổi.
 - + Khi $i = 1$: `leader[1] == leader[1]` -> `leader[1]` gán bằng `leader[3]` (3). Lúc này `leader[1]` trở thành 3.
 - + Khi $i = 2$: So sánh `leader[2]` với `leader[1]` (bây giờ đã là 3). Vì 2 khác 3 nên `leader[2]` không được cập nhật và giữ nguyên là 2.
 - + Mảng thu được sau lệnh này: `leader = [0, 3, 2, 3]`.
 - Kết quả : `find(2) != find(3)` (trả về hai giá trị khác nhau mặc dù thuộc cùng một tập hợp)

Bài 2:

- (a) Số thành phần liên thông là 1 : vì phần tử 0 được nối với tất cả các phần tử còn lại tạo thành một thành phần liên thông duy nhất
- (b) Phép `union(0,k)` cần cập nhật đại diện cho k phần tử. Tổng số lần cập nhật mảng qua $n - 1$ phép `union` là: $1 + 2 + 3 + \dots + (n - 1) = n \cdot (n - 1) / 2$
- (c) Do `parent[q]` không đổi , các nút nối tiếp nhau tạo thành một dạng đường thẳng dài n nút ($1 \rightarrow 2 \rightarrow \dots \rightarrow n - 1$) . Thao tác `find(0)` phải duyệt từ nút 0 đến nút gốc $n - 1$ nên mất $O(n)$.
- (d) Chỉ mất $O(1)$ vì cây chứa nút 0 luôn có kích thước lớn hơn hoặc bằng các cây đơn lẻ khác , nên nút 0 luôn là gốc . Tất cả các nút còn lại đều trở trực tiếp về 0 . Khi gọi `find(0)` , chương trình kiểm tra `parent[0]` và dừng lại ngay lập tức sau 1 lần truy cập.

Bài 3:

Các cặp (p, q) thỏa mãn là $(4, 0)$; $(4, 5)$; $(6, 4)$; $(8, 9)$. Do weighted quick union là cây nhỏ hơn phải nối gốc của nó vào gốc của cây lớn hơn .

- Cặp (0, 4) sai do khi xét đến số nút bằng nhau thì 4 phải được nối vào 0 -> sai.
- Cặp (2,3) sai do khi $\text{find}(2) = 4$, $\text{find}(3) = 3$ và số nút ở phần tử 2 lớn hơn nên khi $\text{union}(2,3)$ thì 3 phải được nối mũi tên đến 4.
- Cặp (5,7) cũng tương tự như cặp (2,3) nếu $\text{union}(5,7)$ thì 7 phải được nối mũi tên đến 4.
- Cặp (5,0) sai do trong đề bài 5,0 không được nối với nhau .

Bài 4:

$\text{parent}[8]$ có thể nhận 0,4,8

Từ đề bài ta có thể xác định được 3 thành phần liên thông sau: nhóm 1 { 0, 1, 2, 3 } gốc là 0; nhóm 2 { 4, 5, 6, 7 } gốc là 4 ; nhóm 3 { 8, 9 } gốc là 8 .

Nếu không union nhóm 3 thì $\text{parent}[8]$ chính là 8 .

Nếu union 8 với một trong các số trong nhóm 1 thì $\text{parent}[8]$ chính là 0 vì gốc của nhóm 1 là 0 và số nút của nhóm 1 nhiều hơn của nhóm 3 nên gốc nhóm 3 được nối vào gốc của nhóm 1; việc union 8 với nhóm 2 cũng tương tự .

Bài 5:

Theo bài ra thì ta vẽ được một đồ thị liên thông với gốc là 5 . Ta xét thời điểm trước khi 0 được nối vào 5 thì ta có 2 thành phần liên thông gồm { 0, 1, 2, 3, 4, 6, 8 } và {5, 7, 9 } . Theo weighted quick union thì gốc của cây nhỏ hơn phải trở vào gốc của cây lớn hơn nên 0 không thể trở đến 5 nhưng để lại cho 0 trở đến 5 -> đây không thể là biểu diễn parent-link của một cấu trúc dữ liệu weighted quick union .